

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

**MODUL 14
GRAPH**



Disusun Oleh :

NAMA : RAIHAN DZAKY MUFLIH

NIM : 103112430029

Dosen

FAHRUDIN MUKTI WIBOWO

**PROGRAM STUDI STRUKTUR DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025**

A. Dasar Teori

Program ini merupakan implementasi struktur data graf yang merepresentasikan hubungan antar-entitas melalui Node sebagai titik data dan Edge sebagai jalur penghubung dua arah. Dalam sistem ini, informasi dikelola menggunakan metode *Adjacency List* yang efisien, di mana setiap node mencatat daftar tetangga langsungnya untuk menghemat penggunaan memori. Selain menyimpan data, program ini menyediakan dua mekanisme penelusuran utama: Depth First Search (DFS) yang mengeksplorasi setiap cabang secara vertikal hingga mencapai kedalaman maksimal, serta Breadth First Search (BFS) yang memproses seluruh simpul pada level yang sama secara horizontal sebelum bergerak lebih jauh.

B. Guided (berisi screenshot source code & output program disertai penjelasannya)

#graph.h

```
#ifndef GRAPH_H_INCLUDED
#define GRAPH_H_INCLUDED

#include <iostream>
using namespace std;

typedef char infoGraph;

struct ElmNode;
struct ElmEdge;

typedef ElmNode* adrNode;
typedef ElmEdge* adrEdge;

struct ElmNode {
    infoGraph info;
    int visited;
    adrEdge firstEdge;
    adrNode next;
};

struct ElmEdge {
    adrNode node;
    adrEdge next;
};

struct Graph {
    adrNode first;
```

```
};

// Primitif Graph
void CreateGraph(Graph &G);
adrNode AllocateNode(infoGraph X);
adrEdge AllocateEdge(adrNode N);

void InsertNode(Graph &G, infoGraph X);
adrNode FindNode(Graph G, infoGraph X);

void ConnectNode(Graph &G, infoGraph A, infoGraph B);

void PrintInfoGraph(Graph G);

// Traversal
void ResetVisited(Graph &G);
void PrintDFS(Graph &G, adrNode N);
void PrintBFS(Graph &G, adrNode N);

#endif
```

#graph.cpp

```
#include "graph.h"
#include <queue>
#include <stack>

void CreateGraph(Graph &G) {
    G.first = NULL;
}

adrNode AllocateNode(infoGraph X) {
    adrNode P = new ElmNode;
    P->info = X;
    P->visited = 0;
    P->firstEdge = NULL;
    P->next = NULL;
    return P;
}

adrEdge AllocateEdge(adrNode N) {
    adrEdge P = new ElmEdge;
    P->node = N;
```

```

        P->next = NULL;
        return P;
    }

void InsertNode(Graph &G, infoGraph X) {
    adrNode P = AllocateNode(X);
    P->next = G.first;
    G.first = P;
}

adrNode FindNode(Graph G, infoGraph X) {
    adrNode P = G.first;
    while (P != NULL) {
        if (P->info == X)
            return P;
        P = P->next;
    }
    return NULL;
}

void ConnectNode(Graph &G, infoGraph A, infoGraph B) {
    adrNode N1 = FindNode(G, A);
    adrNode N2 = FindNode(G, B);

    if (N1 == NULL || N2 == NULL) {
        cout << "Node tidak ditemukan!\n";
        return;
    }

    // Buat edge dari N1 ke N2
    adrEdge E1 = AllocateEdge(N2);
    E1->next = N1->firstEdge;
    N1->firstEdge = E1;

    // Karena undirected -> buat edge balik
    adrEdge E2 = AllocateEdge(N1);
    E2->next = N2->firstEdge;
    N2->firstEdge = E2;
}

void PrintInfoGraph(Graph G) {
    adrNode P = G.first;
    while (P != NULL) {

```

```

        cout << P->info << " -> ";
        adrEdge E = P->firstEdge;
        while (E != NULL) {
            cout << E->node->info << " ";
            E = E->next;
        }
        cout << endl;
        P = P->next;
    }
}

void ResetVisited(Graph &G) {
    adrNode P = G.first;
    while (P != NULL) {
        P->visited = 0;
        P = P->next;
    }
}

void PrintDFS(Graph &G, adrNode N) {
    if (N == NULL)
        return;

    N->visited = 1;
    cout << N->info << " ";

    adrEdge E = N->firstEdge;
    while (E != NULL) {
        if (E->node->visited == 0) {
            PrintDFS(G, E->node);
        }
        E = E->next;
    }
}

void PrintBFS(Graph &G, adrNode N) {
    if (N == NULL)
        return;

    queue<adrNode> Q;
    Q.push(N);

    while (!Q.empty()) {

```

```

        adrNode curr = Q.front();
        Q.pop();

        if (curr->visited == 0) {
            curr->visited = 1;
            cout << curr->info << " ";

            adrEdge E = curr->firstEdge;
            while (E != NULL) {
                if (E->node->visited == 0) {
                    Q.push(E->node);
                }
                E = E->next;
            }
        }
    }
}

```

#main.cpp

```

#include "graph.h"
#include "graph.cpp"
#include <iostream>
using namespace std;

int main() {
    Graph G;
    CreateGraph(G);

    // Tambah node
    InsertNode(G, 'A');
    InsertNode(G, 'B');
    InsertNode(G, 'C');
    InsertNode(G, 'D');
    InsertNode(G, 'E');

    // Hubungkan node (Graph tidak berarah)
    ConnectNode(G, 'A', 'B');
    ConnectNode(G, 'A', 'C');
    ConnectNode(G, 'B', 'D');
    ConnectNode(G, 'C', 'E');
}

```

```

    cout << "=== Struktur Graph ===\n";
    PrintInfoGraph(G);

    cout << "\n=== DFS dari Node A ===\n";
    ResetVisited(G);
    PrintDFS(G, FindNode(G, 'A'));

    cout << "\n\n=== BFS dari Node A ===\n";
    ResetVisited(G);
    PrintBFS(G, FindNode(G, 'A'));

    cout << endl;

    return 0;
}

```

Screenshots Output

```

=== Struktur Graph ===
E -> C
D -> B
C -> E A
B -> D A
A -> C B

=== DFS dari Node A ===
A C E B D

=== BFS dari Node A ===
A C B E D

[Done] exited with code=0 in 3.517 seconds

```

Deskripsi:

Program ini merupakan implementasi struktur data graf tak berarah (undirected graph) menggunakan adjacency list yang memungkinkan pengguna untuk membangun, mengelola, serta menelusuri jaringan data secara sistematis. Selain menyediakan fungsi dasar seperti pembuatan simpul (node), penyambungan sisi (edge), dan pencarian data, program ini dilengkapi dengan mekanisme pengaturan ulang status kunjungan guna menjamin akurasi proses navigasi. Untuk keperluan analisis keterhubungan antar simpul, program mengimplementasikan dua algoritma penelusuran utama, yaitu Depth First Search (DFS) untuk eksplorasi jalur secara mendalam dan Breadth First Search (BFS) untuk eksplorasi jalur secara melebar atau per tingkat.

- C. Unguided/Tugas (berisi screenshot source code & output program disertai penjelasannya)

#graph.h

```
#ifndef GRAPH_H
#define GRAPH_H

#include <iostream>
using namespace std;

typedef char infoGraph;
typedef struct ElmNode *adrNode;
typedef struct ElmEdge *adrEdge;

struct ElmNode {
    infoGraph info;
    int visited;
    adrEdge firstEdge;
    adrNode next;
};

struct ElmEdge {
    adrNode node;
    adrEdge next;
};

struct Graph {
    adrNode first;
};

void CreateGraph(Graph &G);
adrNode InsertNode(Graph &G, infoGraph X);
void ConnectNode(adrNode N1, adrNode N2);
void PrintInfoGraph(Graph G);
void ResetVisited(Graph &G);
void PrintDFS(Graph G, adrNode N);
void PrintBFS(Graph G, adrNode N);

#endif
```

#graph.cpp


```

#include "graph.h"

void CreateGraph(Graph &G) {
    G.first = NULL;
}

adrNode InsertNode(Graph &G, infoGraph X) {
    adrNode P = new ElmNode;
    P->info = X;
    P->visited = 0;
    P->firstEdge = NULL;
    P->next = NULL;

    if (G.first == NULL) {
        G.first = P;
    } else {
        adrNode Q = G.first;
        while (Q->next != NULL) {
            Q = Q->next;
        }
        Q->next = P;
    }
    return P;
}

void ConnectNode(adrNode N1, adrNode N2) {
    adrEdge E1 = new ElmEdge;
    E1->node = N2;
    E1->next = N1->firstEdge;
    N1->firstEdge = E1;

    adrEdge E2 = new ElmEdge;
    E2->node = N1;
    E2->next = N2->firstEdge;
    N2->firstEdge = E2;
}

void PrintInfoGraph(Graph G) {
    adrNode P = G.first;

```

```

        while (P != NULL) {
            cout << P->info << " -> ";
            adrEdge E = P->firstEdge;
            while (E != NULL) {
                cout << E->node->info << " ";
                E = E->next;
            }
            cout << endl;
            P = P->next;
        }
    }

void ResetVisited(Graph &G) {
    adrNode P = G.first;
    while (P != NULL) {
        P->visited = 0;
        P = P->next;
    }
}

void PrintDFS(Graph G, adrNode N) {
    if (N == NULL || N->visited == 1) return;

    cout << N->info << " ";
    N->visited = 1;

    adrEdge E = N->firstEdge;
    while (E != NULL) {
        PrintDFS(G, E->node);
        E = E->next;
    }
}

void PrintBFS(Graph G, adrNode N) {
    if (N == NULL) return;

    adrNode queue[100];
    int front = 0, rear = 0;

    queue[rear++] = N;

```

```

    N->visited = 1;

    while (front < rear) {
        adrNode P = queue[front++];
        cout << P->info << " ";

        adrEdge E = P->firstEdge;
        while (E != NULL) {

            if (E->node->visited == 0) {
                E->node->visited = 1;
                queue[rear++] = E->node;
            }
            E = E->next;
        }
    }
}

```

#main.cpp

```

#include "graph.h"
#include "graph.cpp"

int main() {
    Graph G;
    CreateGraph(G);

    adrNode A = InsertNode(G, 'A');
    adrNode B = InsertNode(G, 'B');
    adrNode C = InsertNode(G, 'C');
    adrNode D = InsertNode(G, 'D');
    adrNode E = InsertNode(G, 'E');
    ConnectNode(A, B);
    ConnectNode(A, C);
    ConnectNode(B, D);
    ConnectNode(C, D);
    ConnectNode(D, E);

    cout << "=== GRAPH ===" << endl;
    PrintInfoGraph(G);

    cout << "\nDFS mulai dari A: ";
}

```

```

    ResetVisited(G);
    PrintDFS(G, A);

    cout << "\n\nBFS mulai dari A: ";
    ResetVisited(G);
    PrintBFS(G, A);

    cout << endl;
    return 0;
}

```

Screenshots Output 1

```

=== GRAPH ===
A -> C B
B -> D A
C -> D A
D -> E C B
E -> D

```

Screenshots Output 2

```

=== GRAPH ===
A -> C B
B -> D A
C -> D A
D -> E C B
E -> D

DFS mulai dari A: A C D E B

```

Screenshots Output 3

```

=== GRAPH ===
A -> C B
B -> D A
C -> D A |
D -> E C B
E -> D

DFS mulai dari A: A C D E B

BFS mulai dari A: A C B D E

```

Deskripsi:

Program ini merupakan simulasi peta jaringan sederhana di mana Node diibaratkan sebagai kota dan Edge sebagai jalan raya dua arah yang menghubungkan kota-kota tersebut. Di dalamnya, kamu bisa membangun kota baru, menyambungkan satu kota dengan kota lainnya, serta melihat daftar seluruh rute yang tersedia. Selain itu, program ini dilengkapi dengan dua cara cerdas untuk menelusuri rute: DFS yang bekerja seperti petualang yang menyusuri satu jalur sedalam mungkin hingga mentok baru kemudian berbelok, dan BFS yang bekerja seperti riak air yang menyebar rata ke seluruh tetangga terdekat terlebih dahulu sebelum menjangkau area yang lebih jauh.

D. Kesimpulan

Kesimpulan dari program dan teori graf yang telah dibahas adalah bahwa Graph merupakan struktur data non-linear yang sangat efektif untuk memodelkan hubungan kompleks antar entitas melalui kumpulan simpul (Node) dan jalur penghubung (Edge). Melalui implementasi Adjacency List, program ini mampu mengelola data secara efisien dengan hanya menyimpan hubungan yang benar-benar ada, serta menyediakan dua metode navigasi fundamental: Depth First Search (DFS) untuk eksplorasi jalur secara mendalam dan Breadth First Search (BFS) untuk penelusuran berbasis level atau jarak terdekat. Secara keseluruhan, struktur ini menjadi fondasi penting dalam menyelesaikan berbagai persoalan teknis, mulai dari pemetaan rute perjalanan hingga analisis jaringan sosial.

E. Referensi

Grandjean, Martin (2016). "[A social network analysis of Twitter: Mapping the digital humanities community](https://doi.org/10.1080/23311983.2016.1171458)". *Cogent Arts & Humanities*. **3** (1) 1171458. doi:10.1080/23311983.2016.1171458. Archived from the original on 2021-03-02. Retrieved 2019-09-16.

Lewis, John (2013), *Java Software Structures* (4th ed.), Pearson, p. 405, ISBN 978-0-13-325012-1

Strang, Gilbert (2005), *Linear Algebra and Its Applications* (4th ed.), Brooks Cole, ISBN 978-0-03-010567-8